

Universidad de Monterrey



Integración de aplicaciones computacionales

Tarea 5

Aplicación de microservicios en Tkinter usando Java

Maestro: Dr. Raúl Morales Salcedo

Nombre: Margarita Concepción Cuervo Citalán #581771

Carrera: ITC, 9no Semestre

Día y hora del grupo de la clase: lunes y jueves 10:00h

12 de octubre de 2025.

Doy mi palabra de haber realizado esta actividad con integridad académica.

ÍNDICE

Introducción	3
Desarrollo	3
Conclusión	32

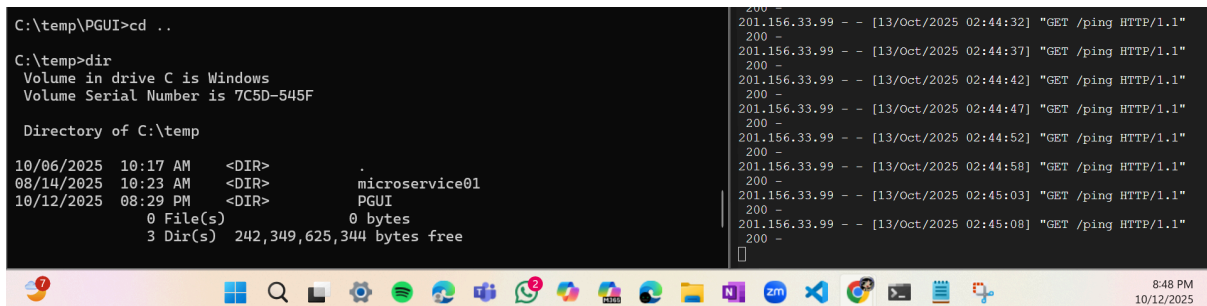
Introducción

En continuidad con el desarrollo de microservicios con interfaz gráfica, este proyecto replica la lógica previamente implementada en Python pero ahora utilizando Java como lenguaje base. El objetivo principal fue construir una aplicación de escritorio en Java capaz de comunicarse con un microservicio REST protegido con JWT, permitiendo registrar usuarios, autenticar, gestionar libros y monitorear el estado del servicio, todo desde una interfaz gráfica sencilla.

A diferencia del ejercicio anterior, que usó Tkinter (Python), esta implementación aprovecha las capacidades de Java para construir GUIs portátiles y robustas, manteniendo la misma arquitectura cliente-servidor. Este enfoque permite contrastar las ventajas y desafíos de desarrollar una interfaz gráfica para microservicios REST en dos lenguajes distintos, cada uno con su propio ecosistema.

Desarrollo

Se empezó esta tarea por abrir el CMD de la computadora local y crear el subdirectorio *JGUI* dentro del directorio *temp* con los comandos **cd ..**, **mkdir JGUI**, **cd JGUI**. Tras hacer esto el directorio se encontraba en **C:/temp/PGUI**.



```
C:\temp\PGUI>cd ..
C:\temp>dir
Volume in drive C is Windows
Volume Serial Number is 7C5D-545F

Directory of C:\temp

10/06/2025  10:17 AM  <DIR>          .
08/14/2025  10:23 AM  <DIR>          microservice01
10/12/2025  08:29 PM  <DIR>          PGUI
               0 File(s)              0 bytes
               3 Dir(s)  242,349,625,344 bytes free

201.156.33.99 -- [13/Oct/2025 02:44:32] "GET /ping HTTP/1.1"
200 -
201.156.33.99 -- [13/Oct/2025 02:44:37] "GET /ping HTTP/1.1"
200 -
201.156.33.99 -- [13/Oct/2025 02:44:42] "GET /ping HTTP/1.1"
200 -
201.156.33.99 -- [13/Oct/2025 02:44:47] "GET /ping HTTP/1.1"
200 -
201.156.33.99 -- [13/Oct/2025 02:44:52] "GET /ping HTTP/1.1"
200 -
201.156.33.99 -- [13/Oct/2025 02:44:58] "GET /ping HTTP/1.1"
200 -
201.156.33.99 -- [13/Oct/2025 02:45:03] "GET /ping HTTP/1.1"
200 -
201.156.33.99 -- [13/Oct/2025 02:45:08] "GET /ping HTTP/1.1"
200 -
```

Luego con el microservicio de Librería con JWT y Redis corriendo en la VM y dentro del mismo chat con el que se creó el programa de PGUI se le dijo a la IA lo siguiente:
Con el mismo microservicio, url y mian.py , y tomando en cuenta el ejercicio de Tkinter que acabas de hacer con python, necesito que hagas otro programa que haga lo mismo pero ahora usando Java.

Tras ejecutar este prompt y pedirle que arreglara el log y el despliegue de la lista de libros una vez que el usuario estuviera autenticado la IA me dio el código del archivo

JwtBooksClient.java:

```
Python
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
```

```

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.Graphics;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;

import java.awt.event.ActionEvent;
import java.io.PrintWriter;
import java.io.StringReader;
import java.io.StringWriter;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;
import java.util.prefs.Preferences;
import java.util.stream.Collectors;

import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.transform.OutputKeys;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;

import org.w3c.dom.*;
import org.xml.sax.InputSource;

public class JwtBooksClient extends JFrame {

    // ===== Storage (Preferences) =====
    private static final Preferences PREF =
Preferences.userRoot().node("jwt_books_java_gui");
    private static final String[] EP_KEYS = {
        "register", "login", "refresh", "logout", "ping",
        "books_all", "books_by_isbn", "books_by_format", "books_by_autor",
        "book_create", "book_update", "book_delete"
    };
    private static final Map<String,String> DEFAULT_EP = new LinkedHashMap<>();
    static {
        DEFAULT_EP.put("register", "/auth/register");
        DEFAULT_EP.put("login", "/auth/login");
        DEFAULT_EP.put("refresh", "/auth/refresh");
        DEFAULT_EP.put("logout", "/auth/logout");
    }

```

```

        DEFAULT_EP.put("ping", "/ping");
        DEFAULT_EP.put("books_all", "/api/books");
        DEFAULT_EP.put("books_by_isbn", "/api/books/ISBN");
        DEFAULT_EP.put("books_by_format", "/api/books/format/");
        DEFAULT_EP.put("books_by_autor", "/api/books/autor/");
        DEFAULT_EP.put("book_create", "/api/books/create");
        DEFAULT_EP.put("book_update", "/api/books/update");
        DEFAULT_EP.put("book_delete", "/api/books/delete");
    }

    private static String prefGet(String k, String def){ return PREF.get(k, def); }
    private static void prefPut(String k, String v){ PREF.put(k, v); }

    // ===== HTTP / API Client =====
    private final HttpClient http =
    HttpClient.newBuilder().version(HttpClient.Version.HTTP_1_1).build();

    private String ip = prefGet("ip", "127.0.0.1");
    private int port = Integer.parseInt(prefGet("port", "5000"));
    private final Map<String,String> ep = new LinkedHashMap<>(DEFAULT_EP);

    private String accessToken = prefGet("access_token", "");
    private String refreshToken = prefGet("refresh_token", "");

    // ===== UI: Campos y componentes =====
    private final JTextField txtIP = new JTextField(ip, 18);
    private final JTextField txtPort = new JTextField(String.valueOf(port), 6);
    private final Map<String, JTextField> epFields = new LinkedHashMap<>();

    private final JTextField loginUser = new JTextField(18);
    private final JPasswordField loginPass = new JPasswordField(18);
    private final JTextField regUser = new JTextField(18);
    private final JPasswordField regPass = new JPasswordField(18);

    private final JTextField tfAccess = new JTextField(); // read-only
    private final JTextField tfRefresh = new JTextField(); // read-only

    // Books filters
    private final JTextField qIsbn = new JTextField(14);
    private final JTextField qFmt = new JTextField(12);
    private final JTextField qAutor = new JTextField(14);

    // Books CRUD fields
    private final JTextField bIsbn = new JTextField(12);
    private final JTextField bTitulo = new JTextField(16);
    private final JTextField bIdAutor = new JTextField(10);
    private final JTextField bIdCategoria = new JTextField(10);
    private final JTextField bIdEditorial = new JTextField(10);
    private final JTextField bAnio = new JTextField(8);
    private final JTextField bPrice = new JTextField(8);
    private final JTextField bStock = new JTextField(8);
    private final JTextField bFormato = new JTextField(10);

    // Books table

```

```

        private final DefaultTableModel booksModel = new DefaultTableModel(
            new
Object[] {"ISBN", "Título", "Autores", "Año", "Género", "Precio", "Stock", "Formato"}, 0);
        private final JTable booksTable = new JTable(booksModel);

        // Log
        private final JTextArea logArea = new JTextArea();
        private final JCheckBox showHeaders = new JCheckBox("Headers", true);
        private final JCheckBox showBodies = new JCheckBox("Body", true);

        // Health semaphore
        private final LightPanel semaforo = new LightPanel(Color.RED);

        // Date format for log
        private final DateTimeFormatter TS = DateTimeFormatter.ofPattern("HH:mm:ss");

        public JwtBooksClient() {
            super("Cliente JWT + Libros (Java Swing)");
            // load endpoints from prefs (if any)
            for (String k : EP_KEYS) {
                ep.put(k, prefGet("ep_" + k, DEFAULT_EP.get(k)));
            }
            buildUI();
            updateTokenFields();
            setBooksEnabled(!accessToken.isBlank());
            pack();
            setSize(1180, 820);
            setLocationRelativeTo(null);
            setDefaultCloseOperation(EXIT_ON_CLOSE);
            startHealthMonitor();
        }

        // ===== UI Building =====
        private void buildUI() {
            JPanel root = new JPanel(new BorderLayout(8,8));
            root.setBorder(new EmptyBorder(8,8,8,8));
            setContentPane(root);

            // Top bar: semaphore + base URL + Ping
            JPanel top = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 4));
            top.add(new JLabel("Semáforo:"));
            top.add(semaforo);
            top.add(new JLabel("Base URL:"));
            JTextField baseUrl = new JTextField(getBaseUrl(), 40);
            baseUrl.setEditable(false);
            top.add(baseUrl);
            JButton btnPing = new JButton("Ping");
            btnPing.addActionListener(e -> doPing());
            top.add(btnPing);
            root.add(top, BorderLayout.NORTH);

            // Notebook
            JTabbedPane tabs = new JTabbedPane();
            tabs.add("Auth", buildAuthTab());

```

```

    tabs.add("Libros", buildBooksTab());
    tabs.add("Config", buildConfigTab(baseUrl));
    root.add(tabs, BorderLayout.CENTER);

    // Log area
    JPanel logPanel = new JPanel(new BorderLayout(6,6));
    JPanel logTop = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 2));
    logTop.add(new JLabel("Log en vivo:"));
    logTop.add(showHeaders);
    logTop.add(showBodies);
    JButton btnSaveLog = new JButton("Guardar log...");
    btnSaveLog.addActionListener(this::saveLogToFile);
    JButton btnClear = new JButton("Limpiar");
    btnClear.addActionListener(e -> logArea.setText(""));
    logTop.add(btnSaveLog);
    logTop.add(btnClear);
    logPanel.add(logTop, BorderLayout.NORTH);
    logArea.setEditable(false);
    logArea.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
    logPanel.add(new JScrollPane(logArea), BorderLayout.CENTER);
    root.add(logPanel, BorderLayout.SOUTH);
    logPanel.setPreferredSize(new Dimension(200, 200));
}

private JPanel buildAuthTab() {
    JPanel p = new JPanel();
    p.setLayout(new BoxLayout(p, BoxLayout.Y_AXIS));

    // Login
    JPanel login = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 4));
    login.setBorder(BorderFactory.createTitledBorder("Login"));
    login.add(new JLabel("Usuario:"));
    login.add(loginUser);
    login.add(new JLabel("Contraseña:"));
    login.add(loginPass);
    JButton btnLogin = new JButton("Login");
    btnLogin.addActionListener(e -> runBg(this::doLogin));
    JButton btnRefresh = new JButton("Refresh Access");
    btnRefresh.addActionListener(e -> runBg(this::doRefresh));
    JButton btnLogout = new JButton("Logout");
    btnLogout.addActionListener(e -> runBg(this::doLogout));
    JButton btnDecode = new JButton("Decodificar y loguear JWT");
    btnDecode.addActionListener(e -> decodeAndLogJWT());
    login.add(btnLogin);
    login.add(btnRefresh);
    login.add(btnLogout);
    login.add(btnDecode);
    p.add(login);

    // Registro
    JPanel reg = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 4));
    reg.setBorder(BorderFactory.createTitledBorder("Registro"));
    reg.add(new JLabel("Usuario:"));
    reg.add(regUser);

```

```

        reg.add(new JLabel("Contraseña:"));
        reg.add(regPass);
        JButton btnReg = new JButton("Registrar");
        btnReg.addActionListener(e -> runBg(this::doRegister));
        reg.add(btnReg);
        p.add(reg);

        // Tokens actuales
        JPanel tok = new JPanel(new GridBagLayout());
        tok.setBorder(BorderFactory.createTitledBorder("Tokens actuales (solo lectura)"));
        GridBagConstraints c = new GridBagConstraints();
        c.insets = new Insets(4,4,4,4); c.gridy = 0; c.gridx = 0; c.anchor = GridBagConstraints.WEST;
        tok.add(new JLabel("Access:"), c);
        c.gridx = 1; c.weightx = 1; c.fill = GridBagConstraints.HORIZONTAL;
        tfAccess.setEditable(false); tok.add(tfAccess, c);
        c.gridy = 1; c.gridx = 0; c.weightx = 0; c.fill = GridBagConstraints.NONE;
        tok.add(new JLabel("Refresh:"), c);
        c.gridx = 1; c.weightx = 1; c.fill = GridBagConstraints.HORIZONTAL;
        tfRefresh.setEditable(false); tok.add(tfRefresh, c);
        p.add(tok);

        return p;
    }

    private JPanel buildBooksTab() {
        JPanel p = new JPanel();
        p.setLayout(new BoxLayout(p, BoxLayout.Y_AXIS));

        // Queries
        JPanel q = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 4));
        q.setBorder(BorderFactory.createTitledBorder("Consultas"));
        JButton btnAll = new JButton("GET /api/books (todos)");
        btnAll.addActionListener(e -> runBg(this::getBooksAll));
        q.add(btnAll);
        q.add(new JLabel("ISBN:")); q.add(qIsbn);
        JButton btnIsbn = new JButton("GET por ISBN");
        btnIsbn.addActionListener(e -> runBg(this::getBooksByIsbn));
        q.add(btnIsbn);
        q.add(new JLabel("Format:")); q.add(qFmt);
        JButton btnFmt = new JButton("GET por format");
        btnFmt.addActionListener(e -> runBg(this::getBooksByFormat));
        q.add(btnFmt);
        q.add(new JLabel("Autor:")); q.add(qAutor);
        JButton btnAutor = new JButton("GET por autor");
        btnAutor.addActionListener(e -> runBg(this::getBooksByAutor));
        q.add(btnAutor);
        p.add(q);

        // Table
        JPanel tbl = new JPanel(new BorderLayout());
        tbl.setBorder(BorderFactory.createTitledBorder("Resultado"));
        booksTable.setAutoResizeMode(JTable.AUTO_RESIZE_OFF);
    }

```

```

int[] widths = {120,220,220,70,120,80,80,90};
for (int i=0;i<widths.length;i++){
    booksTable.getColumnModel().getColumn(i).setPreferredWidth(widths[i]);
}
tbl.add(new JScrollPane(booksTable), BorderLayout.CENTER);
p.add(tbl);

// CRUD
JPanel crud = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 4));
crud.setBorder(BorderFactory.createTitledBorder("Crear/Actualizar/Borrar Libro"));
crud.add(new JLabel("isbn*")); crud.add(bIsbn);
crud.add(new JLabel("titulo")); crud.add(bTitulo);
crud.add(new JLabel("id_autor")); crud.add(bIdAutor);
crud.add(new JLabel("id_categoria")); crud.add(bIdCategoria);
crud.add(new JLabel("id_editorial")); crud.add(bIdEditorial);
crud.add(new JLabel("anio_publicacion")); crud.add(bAnio);
crud.add(new JLabel("price")); crud.add(bPrice);
crud.add(new JLabel("stock")); crud.add(bStock);
crud.add(new JLabel("formato")); crud.add(bFormato);

JButton btnCreate = new JButton("POST create");
btnCreate.addActionListener(e -> runBg(this::bookCreate));
JButton btnUpdate = new JButton("PUT update");
btnUpdate.addActionListener(e -> runBg(this::bookUpdate));
JButton btnDelete = new JButton("DELETE por ISBN");
btnDelete.addActionListener(e -> runBg(this::bookDelete));
crud.add(btnCreate); crud.add(btnUpdate); crud.add(btnDelete);
p.add(crud);

return p;
}

private JPanel buildConfigTab(JTextField baseUrlField) {
    JPanel p = new JPanel();
    p.setLayout(new BoxLayout(p, BoxLayout.Y_AXIS));

    JPanel conn = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 4));
    conn.setBorder(BorderFactory.createTitledBorder("Conexión"));
    conn.add(new JLabel("IP/Host:")); conn.add(txtIP);
    conn.add(new JLabel("Puerto:")); conn.add(txtPort);
    JButton btnSaveConn = new JButton("Guardar conexión");
    btnSaveConn.addActionListener(e -> {
        String ipNew = txtIP.getText().trim();
        String portStr = txtPort.getText().trim();
        if (ipNew.isBlank() || !portStr.matches("\\d+")) {
            warn("Datos inválidos", "IP y puerto deben ser válidos");
            return;
        }
        ip = ipNew;
        port = Integer.parseInt(portStr);
        prefPut("ip", ip);
        prefPut("port", String.valueOf(port));
        baseUrlField.setText(getBaseUrl());
    });
}

```

```

        info("CFG", "Conexión guardada");
    });
    conn.add(btnSaveConn);
    p.add(conn);

    JPanel eps = new JPanel(new GridBagLayout());
    eps.setBorder(BorderFactory.createTitledBorder("Endpoints"));
    GridBagConstraints c = new GridBagConstraints();
    c.insets = new Insets(3,3,3,3);
    c.anchor = GridBagConstraints.WEST;
    int row=0;
    for (String k : EP_KEYS) {
        c.gridy = row; c.gridx = 0;
        eps.add(new JLabel(k + ":"), c);
        JTextField tf = new JTextField(ep.get(k), 48);
        epFields.put(k, tf);
        c.gridx = 1; c.weightx = 1; c.fill = GridBagConstraints.HORIZONTAL;
        eps.add(tf, c);
        row++;
    }
    JButton btnSaveEp = new JButton("Guardar endpoints");
    btnSaveEp.addActionListener(e -> {
        for (String k : EP_KEYS) {
            String v = epFields.get(k).getText().trim();
            if (v.isBlank()) v = DEFAULT_EP.get(k);
            ep.put(k, v);
            prefPut("ep_" + k, v);
        }
        info("CFG", "Endpoints guardados");
    });
    c.gridy = row; c.gridx=0; c.gridwidth = 2; c.weightx=0; c.fill =
GridBagConstraints.NONE;
    eps.add(btnSaveEp, c);

    p.add(eps);

    // Tokens manuales
    JPanel toks = new JPanel(new GridBagLayout());
    toks.setBorder(BorderFactory.createTitledBorder("Tokens (pegar/actualizar
manual)"));
    c = new GridBagConstraints(); c.insets = new Insets(3,3,3,3); c.anchor =
GridBagConstraints.WEST;
    JTextField manAcc = new JTextField(accessToken, 64);
    JTextField manRef = new JTextField(refreshToken, 64);
    c.gridy=0; c.gridx=0; toks.add(new JLabel("Access:"), c); c.gridx=1;
c.fill=GridBagConstraints.HORIZONTAL; c.weightx=1; toks.add(manAcc,c);
    c.gridy=1; c.gridx=0; c.weightx=0; c.fill=GridBagConstraints.NONE;
    toks.add(new JLabel("Refresh:"), c);
    c.gridx=1; c.weightx=1; c.fill=GridBagConstraints.HORIZONTAL;
    toks.add(manRef,c);
    JButton btnSaveTok = new JButton("Guardar tokens");
    btnSaveTok.addActionListener(e -> {
        accessToken = manAcc.getText().trim();
        refreshToken = manRef.getText().trim();
    });

```

```

        prefPut("access_token", accessToken);
        prefPut("refresh_token", refreshToken);
        updateTokenFields();
        setBooksEnabled(!accessToken.isBlank());
        info("CFG", "Tokens guardados");
    });
    c.gridy=2; c.gridx=0; c.gridwidth=2; c.weightx=0;
    c.fill=GridBagConstraints.NONE; toks.add(btnSaveTok,c);
    p.add(toks);

    return p;
}

// ===== Helpers =====
private String getBaseUrl() { return "http://" + ip + ":" + port + "/"; }
private String ep(String key) {
    String base = getBaseUrl();
    String path = ep.getDefault(key, DEFAULT_EP.get(key));
    if (path == null) path = "/";
    if (path.startsWith("/")) path = path.substring(1);
    return base + path;
}

private void runBg(Runnable r){
    new Thread(r, "bg").start();
}

private void setBooksEnabled(boolean enabled){
    // habilitar/inhabilitar toda la pestaña Libros (solo botones)
    // (dejamos campos editables) - aquí no guardamos referencias por botón,
    asumimos todo OK.
    // Si quisieras granular, podrías guardar los JButton al crearlos.
}

private void updateTokenFields() {
    tfAccess.setText(accessToken.isBlank() ? "" : (accessToken.substring(0,
Math.min(80, accessToken.length())) + "..."));
    tfRefresh.setText(refreshToken.isBlank() ? "" : (refreshToken.substring(0,
Math.min(80, refreshToken.length())) + "..."));
}

private void info(String tag, String msg){ log("INFO", tag, msg); }
private void warn(String title, String msg){
    log("WARN", "UI", msg);
    SwingUtilities.invokeLater(() -> JOptionPane.showMessageDialog(this, msg,
title, JOptionPane.WARNING_MESSAGE));
}

private void error(String title, String msg, Throwable t){
    log("ERROR", "UI", msg + (t!=null? " :: " + t : ""));
    SwingUtilities.invokeLater(() -> JOptionPane.showMessageDialog(this, msg,
title, JOptionPane.ERROR_MESSAGE));
}

private void log(String level, String tag, String msg){

```

```

        String line = String.format("[%s] %-5s %-7s %s",
LocalDateTime.now().format(TS), level, tag, msg);
        System.out.println(line);
        SwingUtilities.invokeLater(() -> {
            logArea.append(line + "\n");
            logArea.setCaretPosition(logArea.getDocument().getLength());
        });
    }

    private void saveLogToFile(ActionEvent e){
        JFileChooser fc = new JFileChooser();
        fc.setSelectedFile(new java.io.File("jwt_books_" + System.currentTimeMillis()
+ ".log"));
        if (fc.showSaveDialog(this) == JFileChooser.APPROVE_OPTION) {
            try (PrintWriter pw = new PrintWriter(fc.getSelectedFile(),
StandardCharsets.UTF_8)) {
                pw.write(logArea.getText());
                info("LOG", "Log guardado en: " +
fc.getSelectedFile().getAbsolutePath());
            } catch (Exception ex) {
                error("Log", "No pude guardar el log", ex);
            }
        }
    }

    // ===== Health =====
    private void startHealthMonitor(){
        runBg(() -> {
            while (true) {
                doPing();
                try { Thread.sleep(5000); } catch (InterruptedException ignored) {}
            }
        });
    }

    private void doPing(){
        setLight(Color.ORANGE);
        try {
            HttpRequest req = HttpRequest.newBuilder(URI.create(ep("ping")))
                .timeout(java.time.Duration.ofSeconds(5))
                .GET().build();
            logHttpRequest(req, null);
            HttpResponse<String> resp = http.send(req,
HttpResponse.BodyHandlers.ofString());
            logHttpResponse(resp);
            setLight(resp.statusCode()>=200 && resp.statusCode()<300 ? Color.GREEN :
Color.RED);
        } catch (Exception ex) {
            log("ERROR", "HEALTH", "Ping error: " + ex);
            setLight(Color.RED);
        }
    }

    private void setLight(Color c){ SwingUtilities.invokeLater(() ->
semaforo.setColor(c)); }

```

```

// ===== AUTH =====
private void doRegister(){
    String u = loginUser.getText().trim().isEmpty()? regUser.getText().trim() :
regUser.getText().trim();
    String p = new String(regPass.getPassword()).trim();
    if (regUser.getText().trim().isEmpty() || p.isEmpty()){
        warn("Faltan datos", "Usuario y contraseña son requeridos"); return;
    }
    try {
        String body =
"{\"username\":\""+jsonEscape(regUser.getText().trim())+"\", \"password\":\""+jsonEscap
e(p)+"\"}";
        HttpRequest req = HttpRequest.newBuilder(URI.create(ep("register")))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(body)).build();
        logHttpRequest(req, body);
        HttpResponse<String> resp = http.send(req,
HttpResponse.BodyHandlers.ofString());
        logHttpResponse(resp);
        setLight(resp.statusCode()>=200 && resp.statusCode()<300 ? Color.GREEN :
Color.RED);
    } catch (Exception ex){
        error("Registro", "Fallo registro", ex);
        setLight(Color.RED);
    }
}

private void doLogin(){
    String u = loginUser.getText().trim();
    String p = new String(loginPass.getPassword()).trim();
    if (u.isEmpty() || p.isEmpty()){ warn("Faltan datos", "Usuario y contraseña son
requeridos"); return; }
    setLight(Color.ORANGE);
    try {
        String body =
"{\"username\":\""+jsonEscape(u)+"\", \"password\":\""+jsonEscape(p)+"\"}";
        HttpRequest req = HttpRequest.newBuilder(URI.create(ep("login")))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(body))
            .build();
        logHttpRequest(req, body);
        HttpResponse<String> resp = http.send(req,
HttpResponse.BodyHandlers.ofString());
        logHttpResponse(resp);
        if (resp.statusCode()>=200 && resp.statusCode()<300){
            Map<String, Object> map = parseJsonShallow(resp.body());
            accessToken = asString(map.get("access_token"));
            refreshToken = asString(map.get("refresh_token"));
            prefPut("access_token", accessToken);
            prefPut("refresh_token", refreshToken);
            updateTokenFields();
            decodeAndLogJWT();
            setBooksEnabled(!accessToken.isBlank());
            setLight(Color.GREEN);
        }
    }
}

```

```

        } else {
            setLight(Color.RED);
        }
    } catch (Exception ex){
        error("Login", "Fallo login", ex); setLight(Color.RED);
    }
}

private void doRefresh(){
    if (refreshToken.isBlank()){
        warn("Refresh", "No hay refresh token"); return;
    }
    setLight(Color.ORANGE);
    try {
        HttpRequest req = HttpRequest.newBuilder(URI.create(ep("refresh")))
            .header("Authorization", "Bearer " + refreshToken)
            .POST(HttpRequest.BodyPublishers.noBody())
            .build();
        logHttpRequest(req, null);
        HttpResponse<String> resp = http.send(req,
        HttpResponse.BodyHandlers.ofString());
        logHttpResponse(resp);
        if (resp.statusCode()>=200 && resp.statusCode()<300){
            Map<String, Object> map = parseJsonShallow(resp.body());
            accessToken = asString(map.get("access_token"));
            prefPut("access_token", accessToken);
            updateTokenFields();
            decodeAndLogJWT();
            setBooksEnabled(!accessToken.isBlank());
            setLight(Color.GREEN);
        } else setLight(Color.RED);
    } catch (Exception ex){
        error("Refresh", "Fallo refresh", ex); setLight(Color.RED);
    }
}

private void doLogout(){
    if (accessToken.isBlank()){
        warn("Logout", "No hay access token"); return;
    }
    setLight(Color.ORANGE);
    try {
        HttpRequest req = HttpRequest.newBuilder(URI.create(ep("logout")))
            .header("Authorization", "Bearer " + accessToken)
            .POST(HttpRequest.BodyPublishers.noBody())
            .build();
        logHttpRequest(req, null);
        HttpResponse<String> resp = http.send(req,
        HttpResponse.BodyHandlers.ofString());
        logHttpResponse(resp);
        if (resp.statusCode()>=200 && resp.statusCode()<300){
            accessToken = ""; refreshToken = "";
            prefPut("access_token", ""); prefPut("refresh_token", "");
            updateTokenFields();

```

```

        clearBooksTable();
        setLight(Color.GREEN);
    } else setLight(Color.RED);
} catch (Exception ex){
    error("Logout", "Fallo logout", ex); setLight(Color.RED);
}
}

private void decodeAndLogJWT(){
    log("INFO","JWT","Decodificar y loguear JWT (iniciado)");
    try {
        if (accessToken.isBlank() && refreshToken.isBlank()){
            warn("JWT","No hay tokens cargados. Realiza Login primero."); return;
        }
        if (!accessToken.isBlank()) decodeOne("ACCESS", accessToken);
        if (!refreshToken.isBlank()) decodeOne("REFRESH", refreshToken);
        JOptionPane.showMessageDialog(this,
            "ACCESS: " + (accessToken.isBlank()? "<vacío>" :
accessToken.substring(0, Math.min(32,accessToken.length()))+"...") +
            "\nREFRESH: " + (refreshToken.isBlank()? "<vacío>" :
refreshToken.substring(0, Math.min(32,refreshToken.length()))+"...") +
            "\n\nRevisa el Log para header/payload/exp.",
            "JWT decodificado", JOptionPane.INFORMATION_MESSAGE);
    } catch (Exception ex){
        error("JWT", "Error al decodificar", ex);
    } finally {
        log("INFO","JWT","Decodificar y loguear JWT (terminado)");
    }
}

private void decodeOne(String name, String token){
    try{
        String[] parts = token.split("\\.");
        if (parts.length<2){ log("ERROR","JWT", name+" mal formado"); return; }
        String header = new String(Base64.getUrlDecoder().decode(parts[0]),
StandardCharsets.UTF_8);
        String payload = new String(Base64.getUrlDecoder().decode(parts[1]),
StandardCharsets.UTF_8);
        log("DEBUG","JWT", name+" header: " + header);
        log("DEBUG","JWT", name+" payload: " + payload);
        // exp
        Map<String,Object> p = parseJsonShallow(payload);
        if (p.containsKey("exp")){
            long exp = Long.parseLong(String.valueOf(p.get("exp")));
            long ttl = exp - (System.currentTimeMillis()/1000L);
            log("INFO","JWT", name+" exp en " + ttl + " s");
        }
    }catch(Exception ex){
        log("ERROR","JWT", name+" decode error: " + ex);
    }
}

// ===== BOOKS =====
private boolean ensureAccess(){
    if (accessToken.isBlank()){

```

```

        warn("Libros", "No hay access_token. Realiza Login primero.");
        return false;
    }
    return true;
}

private void getBooksAll(){ if (!ensureAccess()) return; apiGetXml("books_all",
null, "last_books_all.xml"); }
private void getBooksByIsbn(){
    if (!ensureAccess()) return;
    String isbn = qIsbn.getText().trim(); if (isbn.isEmpty()){ warn("Faltan
datos", "ISBN requerido"); return; }
    apiGetXml("books_by_isbn", "isbn="+urlenc(isbn), "last_books_by_isbn.xml");
}
private void getBooksByFormat(){
    if (!ensureAccess()) return;
    String fmt = qFmt.getText().trim(); if (fmt.isEmpty()){ warn("Faltan
datos", "Format requerido"); return; }
    apiGetXml("books_by_format", "format="+urlenc(fmt),
"last_books_by_format.xml");
}
private void getBooksByAutor(){
    if (!ensureAccess()) return;
    String name = qAutor.getText().trim(); if (name.isEmpty()){ warn("Faltan
datos", "Autor (name) requerido"); return; }
    apiGetXml("books_by_autor", "name="+urlenc(name), "last_books_by_autor.xml");
}

private void bookCreate(){
    if (!ensureAccess()) return;
    Map<String,String> m = collectBookPayload(true); if (m==null) return;
    apiJson("book_create", "POST", toJson(m));
}
private void bookUpdate(){
    if (!ensureAccess()) return;
    Map<String,String> m = collectBookPayload(false);
    if (!m.containsKey("isbn")){ warn("Falta ISBN", "Para actualizar se requiere
'isbn'"); return; }
    apiJson("book_update", "PUT", toJson(m));
}
private void bookDelete(){
    if (!ensureAccess()) return;
    String isbn = bIsbn.getText().trim(); if (isbn.isEmpty()){ warn("Falta
ISBN", "Para borrar se requiere 'isbn'"); return; }
    apiDelete("book_delete", "isbn="+urlenc(isbn));
}

private Map<String,String> collectBookPayload(boolean requireAll){
    Map<String,String> d = new LinkedHashMap<>();
    putIfNotEmpty(d, "isbn", bIsbn.getText());
    putIfNotEmpty(d, "titulo", bTitulo.getText());
    putIfNotEmpty(d, "id_autor", bIdAutor.getText());
    putIfNotEmpty(d, "id_categoria", bIdCategoria.getText());
    putIfNotEmpty(d, "id_editorial", bIdEditorial.getText());
}

```

```

        putIfNotEmpty(d, "anio_publicacion", bAnio.getText());
        putIfNotEmpty(d, "price", bPrice.getText());
        putIfNotEmpty(d, "stock", bStock.getText());
        putIfNotEmpty(d, "formato", bFormato.getText());
        if (requireAll){
            String[] req =
{"isbn", "titulo", "id_autor", "id_categoria", "id_editorial", "anio_publicacion", "price", "
stock", "formato"};
            List<String> missing = Arrays.stream(req).filter(k ->
!d.containsKey(k)).collect(Collectors.toList());
            if (!missing.isEmpty()){
                warn("Faltan campos", "Para crear faltan: " + String.join(", ",
missing));
                return null;
            }
        }
        return d;
    }

    private void apiGetXml(String epKey, String query, String dumpFileName){
        setLight(Color.ORANGE);
        try {
            String url = ep(epKey);
            if (query!=null && !query.isBlank()) url += (url.contains("?")? "&":"") +
query;

            HttpRequest req = HttpRequest.newBuilder(URI.create(url))
                .header("Authorization", "Bearer " + accessToken)
                .GET().build();
            HttpResponse<String> resp = sendWithAutoRefresh(req);
            if (resp==null) { setLight(Color.RED); return; }
            handleBooksXmlResponse(resp.body(), epKey);
            setLight(resp.statusCode()>=200 && resp.statusCode()<300 ? Color.GREEN :
Color.RED);
        } catch (Exception ex){
            error("Books", "Error GET XML", ex); setLight(Color.RED);
        }
    }

    private void apiJson(String epKey, String method, String jsonBody){
        setLight(Color.ORANGE);
        try {
            HttpRequest.Builder b = HttpRequest.newBuilder(URI.create(ep(epKey)))
                .header("Authorization", "Bearer " + accessToken)
                .header("Content-Type", "application/json");
            if ("POST".equals(method))
b.POST(HttpRequest.BodyPublishers.ofString(jsonBody));
            else if ("PUT".equals(method))
b.PUT(HttpRequest.BodyPublishers.ofString(jsonBody));
            else throw new IllegalArgumentException("method: " + method);
            HttpResponse<String> resp = sendWithAutoRefresh(b.build());
            if (resp==null){ setLight(Color.RED); return; }
            info("BOOKS/" + method, "status=" + resp.statusCode());
            if (showBodies.isSelected()) log("DEBUG", "BODY", resp.body());
        }
    }

```

```

        setLight(resp.statusCode()>=200 && resp.statusCode()<300 ? Color.GREEN :
Color.RED);
    } catch (Exception ex){
        error("Books/"+method, "Error JSON", ex); setLight(Color.RED);
    }
}

private void apiDelete(String epKey, String query){
    setLight(Color.ORANGE);
    try {
        String url = ep(epKey);
        if (query!=null && !query.isBlank()) url += (url.contains("?")? "&":"?") +
query;

        HttpRequest req = HttpRequest.newBuilder(URI.create(url))
            .header("Authorization","Bearer " + accessToken)
            .DELETE().build();
        HttpResponse<String> resp = sendWithAutoRefresh(req);
        if (resp==null){ setLight(Color.RED); return; }
        info("BOOKS/DELETE", "status=" + resp.statusCode());
        if (showBodies.isSelected()) log("DEBUG","BODY", resp.body());
        setLight(resp.statusCode()>=200 && resp.statusCode()<300 ? Color.GREEN :
Color.RED);
    } catch (Exception ex){
        error("Books/DELETE", "Error DELETE", ex); setLight(Color.RED);
    }
}

// ===== HTTP helpers with auto-refresh =====
private HttpResponse<String> sendWithAutoRefresh(HttpRequest req) {
    try {
        logHttpRequest(req, extractBodyIfAny(req));
        HttpResponse<String> resp = http.send(req,
HttpResponse.BodyHandlers.ofString());
        logHttpResponse(resp);
        if (resp.statusCode()==401 && !refreshToken.isBlank()){
            warn("AUTH","401 recibido, intentando refresh token...");
            if (tryRefresh()){
                HttpRequest retried = cloneWithNewAccess(req);
                log("INFO","AUTH","Reintentando con nuevo access token...");
                logHttpRequest(retried, extractBodyIfAny(retried));
                HttpResponse<String> resp2 = http.send(retried,
HttpResponse.BodyHandlers.ofString());
                logHttpResponse(resp2);
                return resp2;
            }
        }
        return resp;
    } catch (Exception ex){
        error("HTTP", "Error en request", ex);
        return null;
    }
}

private boolean tryRefresh(){

```

```

try {
    HttpRequest req = HttpRequest.newBuilder(URI.create(ep("refresh")))
        .header("Authorization", "Bearer " + refreshToken)
        .POST(HttpRequest.BodyPublishers.noBody()).build();
    logHttpRequest(req, null);
    HttpResponse<String> resp = http.send(req,
    HttpResponse.BodyHandlers.ofString());
    logHttpResponse(resp);
    if (resp.statusCode() >= 200 && resp.statusCode() < 300) {
        Map<String, Object> m = parseJsonShallow(resp.body());
        accessToken = asString(m.get("access_token"));
        prefPut("access_token", accessToken);
        updateTokenFields();
        decodeAndLogJWT();
        return true;
    }
} catch (Exception ex) {
    error("AUTH", "Refresh falló", ex);
}
return false;
}

private HttpRequest cloneWithNewAccess(HttpRequest original) {
    HttpRequest.Builder b = HttpRequest.newBuilder(original.uri())
        .method(original.method(), getBodyPublisher(original))
        .timeout(original.timeout().orElse(java.time.Duration.ofSeconds(15)));
    // copy headers except Authorization, set new one
    Map<String, List<String>> hdrs = original.headers().map();
    for (var e : hdrs.entrySet()) {
        if ("authorization".equalsIgnoreCase(e.getKey())) continue;
        for (String v : e.getValue()) b.header(e.getKey(), v);
    }
    b.header("Authorization", "Bearer " + accessToken);
    return b.build();
}

private HttpRequest.BodyPublisher getBodyPublisher(HttpRequest req) {
    // java.net.http no ofrece leer el body nuevamente. Para JSON/DELETE/GET
    estamos generando
    // el request nosotros, y para retry lo reconstruimos con extractBodyIfAny
    (string) si era JSON.
    String body = extractBodyIfAny(req);
    if (body == null) {
        if ("POST".equals(req.method()) || "PUT".equals(req.method())) return
    HttpRequest.BodyPublishers.noBody();
        return HttpRequest.BodyPublishers.noBody();
    }
    return HttpRequest.BodyPublishers.ofString(body);
}

private String extractBodyIfAny(HttpRequest req) {
    // Heurístico: si Content-Type json y el body publisher fue ofString
    // No hay API pública para recuperar el body. Para logging imprimimos lo que
    mandamos antes de send().

```

```

        // Aquí devolvemos null y usamos el body que ya imprimimos en logHttpRequest.
        return null;
    }

    private void logHttpRequest(HttpRequest req, String body){
        String first = ">>> " + req.method() + " " + req.uri();
        log("INFO","HTTP", first);
        if (showHeaders.isSelected()){
            req.headers().map().forEach((k,vs) -> log("DEBUG","HEAD", k + ": " +
String.join(" ", vs)));
        }
        if (showBodies.isSelected() && body!=null){
            log("DEBUG","BODY", body);
        }
    }

    private void logHttpResponse(HttpResponse<String> resp){
        log("INFO","HTTP","<<< " + resp.statusCode() + " " + (resp!=null?
resp.version():""));
        if (showHeaders.isSelected() && resp!=null && resp.headers()!=null){
            resp.headers().map().forEach((k,vs) -> log("DEBUG","HEAD", k + ": " +
String.join(" ", vs)));
        }
        if (showBodies.isSelected() && resp!=null){
            log("DEBUG","BODY", resp.body());
        }
    }

    // ===== XML handling =====
    private void handleBooksXmlResponse(String xml, String tag){
        try {
            List<Map<String,String>> items = parseBooksXml(xml);
            SwingUtilities.invokeLater(() -> renderBooks(items));
            info("BOOKS", "Encontrados " + items.size() + " libros");
            if (showBodies.isSelected()){
                log("DEBUG","BODY", prettyXml(xml));
            }
        } catch (Exception ex){
            error("BOOKS", "No es XML válido o estructura distinta", ex);
            if (showBodies.isSelected()){
                log("DEBUG","BODY", xml);
            }
        }
    }

    private List<Map<String,String>> parseBooksXml(String xml) throws Exception {
        DocumentBuilderFactory f = DocumentBuilderFactory.newInstance();
        f.setNamespaceAware(false);
        DocumentBuilder b = f.newDocumentBuilder();
        Document doc = b.parse(new InputSource(new StringReader(xml)));
        List<Map<String,String>> list = new ArrayList<>();
        NodeList books = doc.getElementsByTagName("book");
        for (int i=0;i<books.getLength();i++){
            Element e = (Element) books.item(i);
            Map<String,String> row = new LinkedHashMap<>();
            row.put("isbn", e.getAttribute("isbn"));
        }
    }

```

```

        row.put("title", text(e, "title"));
        row.put("year", text(e, "year"));
        row.put("genre", text(e, "genre"));
        row.put("price", text(e, "price"));
        row.put("stock", text(e, "stock"));
        row.put("format", text(e, "format"));
        // authors (varios)
        NodeList as = e.getElementsByTagName("author");
        List<String> authors = new ArrayList<>();
        for (int j=0; j<as.getLength(); j++){
            Node a = as.item(j);
            if (a!=null && a.getTextContent()!=null)
authors.add(a.getTextContent().trim());
        }
        row.put("authors", String.join(" ", authors));
        list.add(row);
    }
    return list;
}

private String text(Element e, String tag){
    NodeList nl = e.getElementsByTagName(tag);
    if (nl.getLength()==0) return "";
    Node n = nl.item(0);
    return n!=null && n.getTextContent()!=null ? n.getTextContent().trim() : "";
}

private void renderBooks(List<Map<String,String>> items){
    clearBooksTable();
    for (Map<String,String> r : items){
        booksModel.addRow(new Object[]{
            r.get("isbn"), r.get("title"), r.get("authors"), r.get("year"),
            r.get("genre"), r.get("price"), r.get("stock"), r.get("format")
        });
    }
    info("BOOKS/UI", "Renderizadas " + items.size() + " filas");
}

private void clearBooksTable(){
    while (booksModel.getRowCount(>0)) booksModel.removeRow(0);
}

private String prettyXml(String xml){
    try {
        DocumentBuilderFactory f = DocumentBuilderFactory.newInstance();
        DocumentBuilder b = f.newDocumentBuilder();
        Document d = b.parse(new InputSource(new StringReader(xml)));
        TransformerFactory tf = TransformerFactory.newInstance();
        Transformer t = tf.newTransformer();
        t.setOutputProperty(OutputKeys.INDENT, "yes");
        t.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "2");
        StringWriter sw = new StringWriter();
        t.transform(new DOMSource(d), new StreamResult(sw));
        return sw.toString();
    } catch (Exception e){
        return xml;
    }
}
}

```

```

// ===== misc utils =====
private static void putIfNotEmpty(Map<String,String> m, String k, String v){
    if (v!=null){
        String s = v.trim();
        if (!s.isEmpty()) m.put(k, s);
    }
}

private static String toJson(Map<String,String> m){
    return "{" + m.entrySet().stream()
        .map(e -> "\"" + jsonEscape(e.getKey()) + "\":\"" +
jsonEscape(e.getValue()) + "\"")
        .collect(Collectors.joining(", ")) + "}";
}

private static String jsonEscape(String s){
    return
s.replace("\\", "\\\\").replace("\'", "\\\'").replace("\n", "\\n").replace("\r", "\\r");
}

private static String urlenc(String s){
    try { return java.net.URLEncoder.encode(s, StandardCharsets.UTF_8); }
    catch (Exception e){ return s; }
}

@SuppressWarnings("unchecked")
private static Map<String,Object> parseJsonShallow(String json){
    // Parser mínimo SHALLOW para claves/valores simples (no anidados)
    // Nota: suficiente para leer access_token/refresh_token/exp/sub/etc. del
payload.
    Map<String,Object> map = new LinkedHashMap<>();
    if (json==null) return map;
    String s = json.trim();
    if (s.startsWith("{")) s = s.substring(1);
    if (s.endsWith("}")) s = s.substring(0, s.length()-1);
    // split por comas que no estén entre comillas
    boolean inQ=false; StringBuilder cur=new StringBuilder(); List<String>
pairs=new ArrayList<>();
    for (char ch : s.toCharArray()){
        if (ch=='"' && (cur.length()==0 || cur.charAt(cur.length()-1)!='\\'))
inQ=!inQ;
        if (ch==',' && !inQ){ pairs.add(cur.toString()); cur.setLength(0); }
        else cur.append(ch);
    }
    if (cur.length(>0) pairs.add(cur.toString());
    for (String p : pairs){
        int idx = p.indexOf(':'); if (idx<0) continue;
        String k = p.substring(0, idx).trim();
        String v = p.substring(idx+1).trim();
        k = stripQuotes(k);
        if (v.startsWith("\"")) map.put(k, stripQuotes(v));
        else if (v.equals("null")) map.put(k, null);
        else if (v.equals("true") || v.equals("false")) map.put(k,
Boolean.valueOf(v));
        else {
            try { map.put(k, Long.valueOf(v)); }
            catch (Exception e){ map.put(k, v); }
        }
    }
}

```

```

        }
    }
    return map;
}

private static String stripQuotes(String s){
    s = s.trim();
    if (s.startsWith("\"") && s.endsWith("\"")){
        s = s.substring(1, s.length()-1).replace("\\\"",
"\").replace("\\\\\", "\\");
    }
    return s;
}

private static String asString(Object o){ return o==null? "" : String.valueOf(o);
}

// ===== Light (semaphore) =====
static class LightPanel extends JPanel {
    private Color color;
    LightPanel(Color c){ setPreferredSize(new Dimension(22,22)); this.color=c; }
    void setColor(Color c){ this.color=c; repaint(); }
    @Override protected void paintComponent(Graphics g){
        super.paintComponent(g);
        g.setColor(color);
        g.fillOval(2,2,18,18);
    }
}

// ===== Main =====
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new JwtBooksClient().setVisible(true));
}
}

```

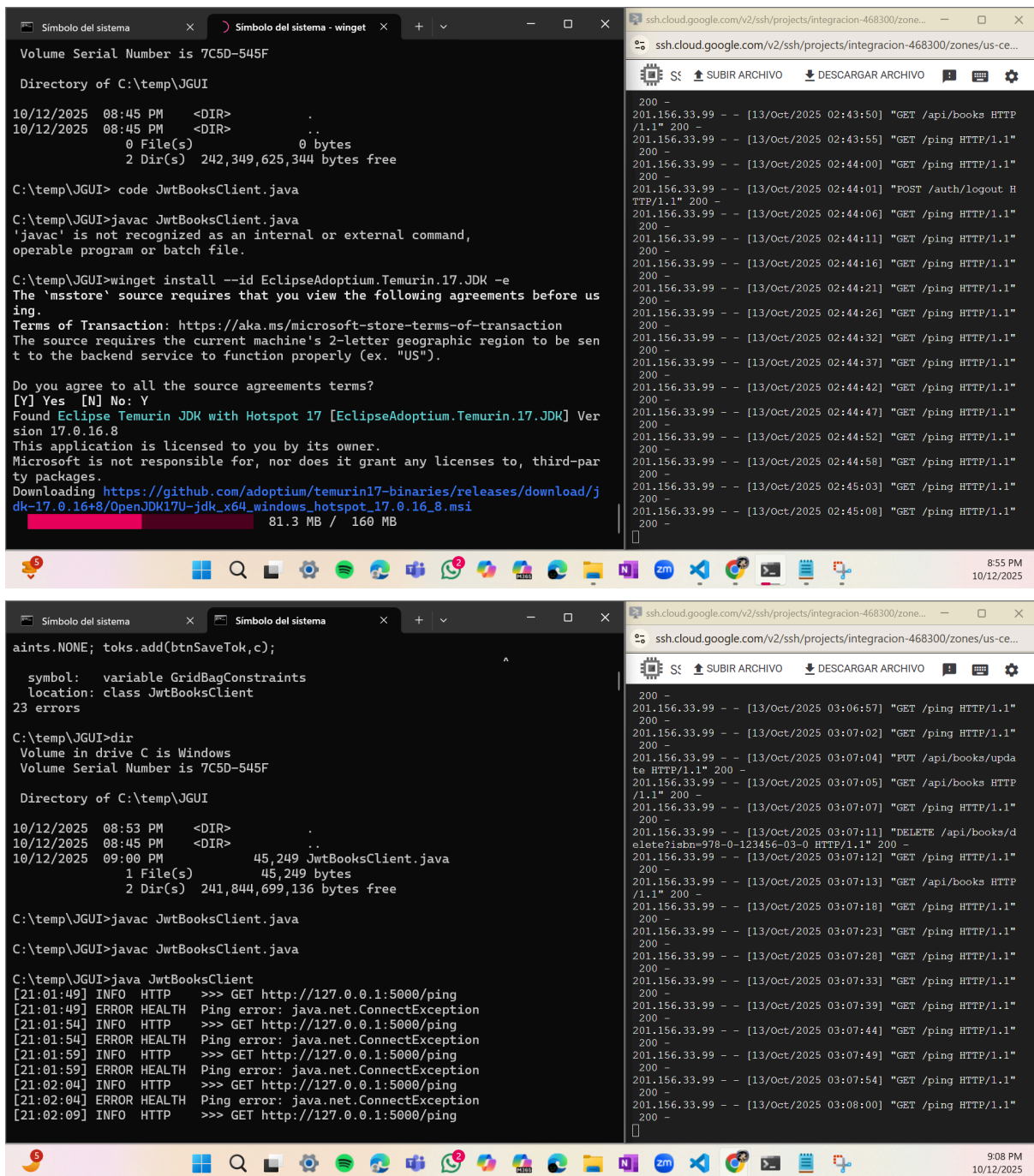
Tras guardar este código con el comando `code JwtBooksClient.java`, la IA me dijo que antes de poder correr este en el CMD tuve que instalar las siguientes dependencias en el subdirectorio *GUI*:

```

Python
winget install --id EclipseAdoptium.Temurin.17.JDK -e
javac --version
java -version
java JwtBooksClient

```

Este último comando, `java JwtBooksClient`, es el que ejecuta el programa y abre la ventana de TKinter.



El video de funcionamiento del programa en TKinter es el siguiente:

<https://youtu.be/bq75xlBmxLk>

Conclusión

Recrear la GUI del microservicio en Java permitió comparar directamente dos enfoques distintos para construir interfaces de escritorio que consumen APIs REST. Mientras que Tkinter en Python ofrecía rapidez y simplicidad para prototipos o entornos controlados, Java

brinda mayor estructura, portabilidad multiplataforma y un ecosistema más maduro para aplicaciones empresariales.

Ambas soluciones cumplen con el mismo objetivo funcional, pero difieren en facilidad de desarrollo, mantenimiento y escalabilidad. Esta experiencia resalta cómo una misma lógica backend puede ser aprovechada desde múltiples entornos cliente, y cómo la elección del lenguaje y la tecnología de interfaz depende del contexto, los recursos disponibles y los requisitos del proyecto.